

AUSTCAIC — AI & ML WING

Class 4: Advanced Machine Learning

Session Date: 22th August, 2026 • Time: 9:00 PM

OVERVIEW

What We'll Cover Today

1. Ensemble Learning & Introduction to XGBoost
2. Feature Importance
3. Feature Selection
4. Hyperparameter Tuning
5. Principal Component Analysis (PCA)
6. Introduction to Reinforcement Learning

RECAP

Quick Recap: Class 3

- We built full supervised pipelines: train/test split, fit, predict, evaluate.
- We got to know about some ML algorithms — Linear & Logistic Regression, Decision Trees K-Means Clustering.
- We learned evaluation metrics: MAE/RMSE for regression, Accuracy/Precision/Recall/Confusion Matrix for classification.
- We learned to spot overfitting vs underfitting.

Today:

We move from foundations to advanced techniques used on real, messy tabular data.

What Is Ensemble Learning?

- A technique that combines predictions from multiple models to produce one better model
- Idea: several models working together often outperform any single model alone
- Reduces the risk of one model's errors, blind spots, or bad luck on the training data
- Three main strategies, each combining models differently: Bagging, Boosting, and Stacking

Bagging

Boosting

Stacking

Bagging vs Boosting vs Stacking

Three ways to combine models — they differ in how the models are trained and how their predictions are merged.

Bagging

e.g. Random Forest

- Trains many models in parallel on random subsets of the data
- Final prediction = average / majority vote
- Reduces variance — more stable, less overfitting

Boosting

e.g. XGBoost

- Trains models sequentially, one after another
- Each new model corrects the previous model's errors
- Reduces bias — turns weak learners into one strong learner

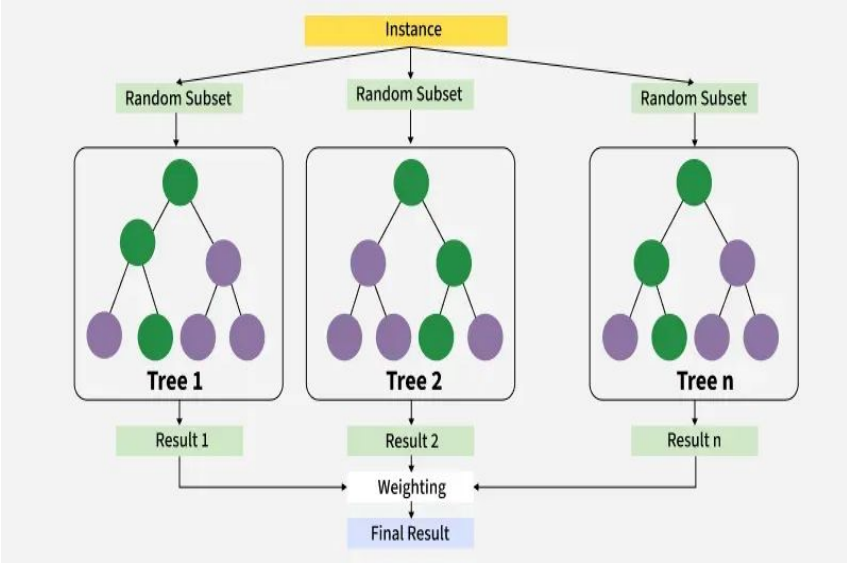
Stacking

e.g. blending Logistic Regression + Tree + KNN

- Trains several different types of models on the same data
- A separate 'meta-model' learns how to best combine their predictions
- Leverages the strengths of different algorithms at once

Introducing XGBoost

- XGBoost = eXtreme Gradient Boosting — an optimized, regularized take on gradient boosting
- Builds decision trees one at a time; each tree fits the residual errors left by the previous trees
- Adds L1/L2 regularization to keep the model from overfitting as it grows
- A go-to choice for structured/tabular data — fast, accurate, and widely used in ML competitions



XGBoost in Scikit-Learn

```
from xgboost import XGBClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

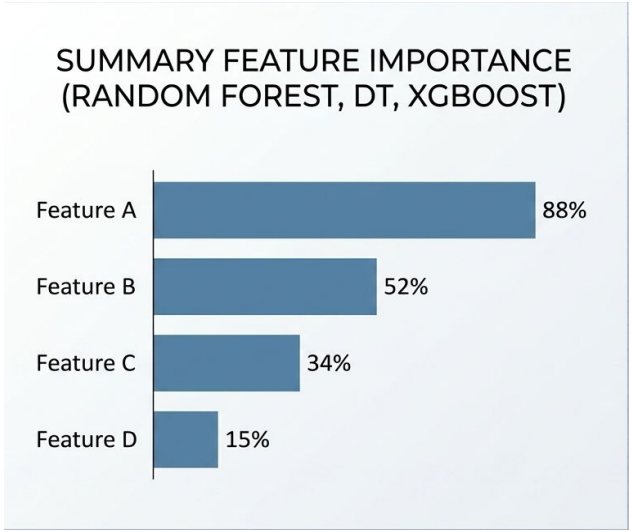
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = XGBClassifier(n_estimators=100, max_depth=3)
model.fit(X_train, y_train)

print(model.score(X_test, y_test))
```

Which Features Actually Matter?

- After training a tree-based model, we can rank features by how much they reduce prediction error
- Helps explain model decisions, catch noisy or irrelevant features, and simplify the pipeline
- Random Forest, Decision Trees, and XGBoost all expose a feature_importances_ attribute



Plotting Feature Importance

```
import pandas as pd
import matplotlib.pyplot as plt

importances = model.feature_importances_
feat_names = load_wine().feature_names

pd.Series(importances, index=feat_names) \
  .sort_values().plot(kind="barh")

plt.title("XGBoost Feature Importance")
plt.show()
```

Three Ways to Select Features

Method	How It Works	Example
Filter	Ranks features by a statistical score, independent of any model	Correlation, Chi-Square, ANOVA
Wrapper	Tries different feature subsets, evaluated using a real model	Recursive Feature Elimination (RFE)
Embedded	Selection happens automatically during model training	Lasso, XGBoost feature_importances_

Goal: fewer, more relevant features → faster training, less overfitting, easier interpretation.

Why Tune?

- **Parameters** are learned by the model during training (e.g. tree weights).
- **Hyperparameters** are set by us beforehand — e.g. max_depth, n_estimators, learning_rate.
- Default settings rarely give the best possible performance on your data.
- Tuning searches the hyperparameter space for the combination that generalizes best.
- **Rule of thumb:** always tune using cross-validation, never the final test set.

Grid Search vs Random Search

Grid Search

- Tries every possible combination in a predefined grid
- Exhaustive — guaranteed to check every option
- Slow: cost grows exponentially as you add parameters

Random Search

- Samples random combinations from the parameter space
- Much faster on large search spaces
- Often finds a near-optimal result in far fewer trials

GridSearchCV in Scikit-Learn

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [2, 3, 4],
    "learning_rate": [0.01, 0.1, 0.2]
}

grid = GridSearchCV(XGBClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)

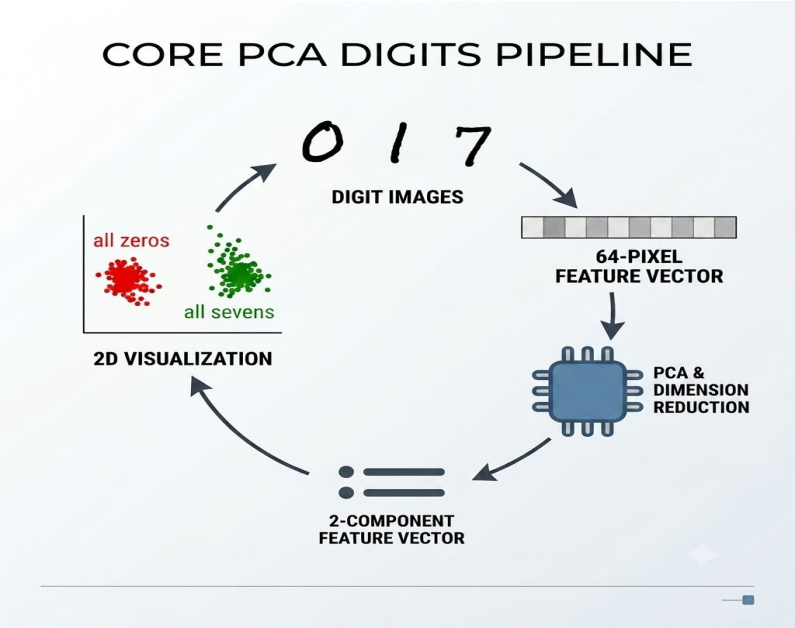
print(grid.best_params_)
```

The Curse of Dimensionality

- Real-world datasets often have dozens or hundreds of features
- Too many features → slower training, more noise, harder to visualize, higher overfitting risk
- Many features are also correlated or redundant with one another
- Dimensionality reduction compresses features while keeping most of the information

How PCA Works

- Finds new axes (principal components) that capture the most variance in the data
- Each component is a weighted combination of the original features
- Components are ranked by how much variance they explain — keep the top few, drop the rest
- Example: compressing 64 pixel features (digits dataset) down to just 2 components to visualize



PCA in Scikit-Learn

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits

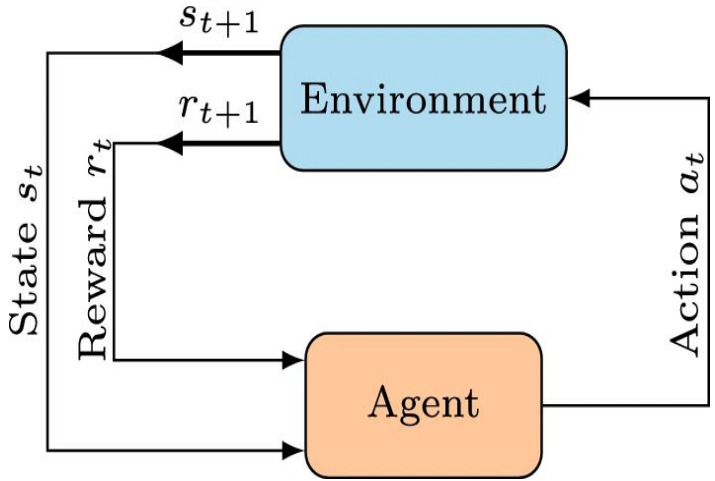
X, y = load_digits(return_X_y=True)

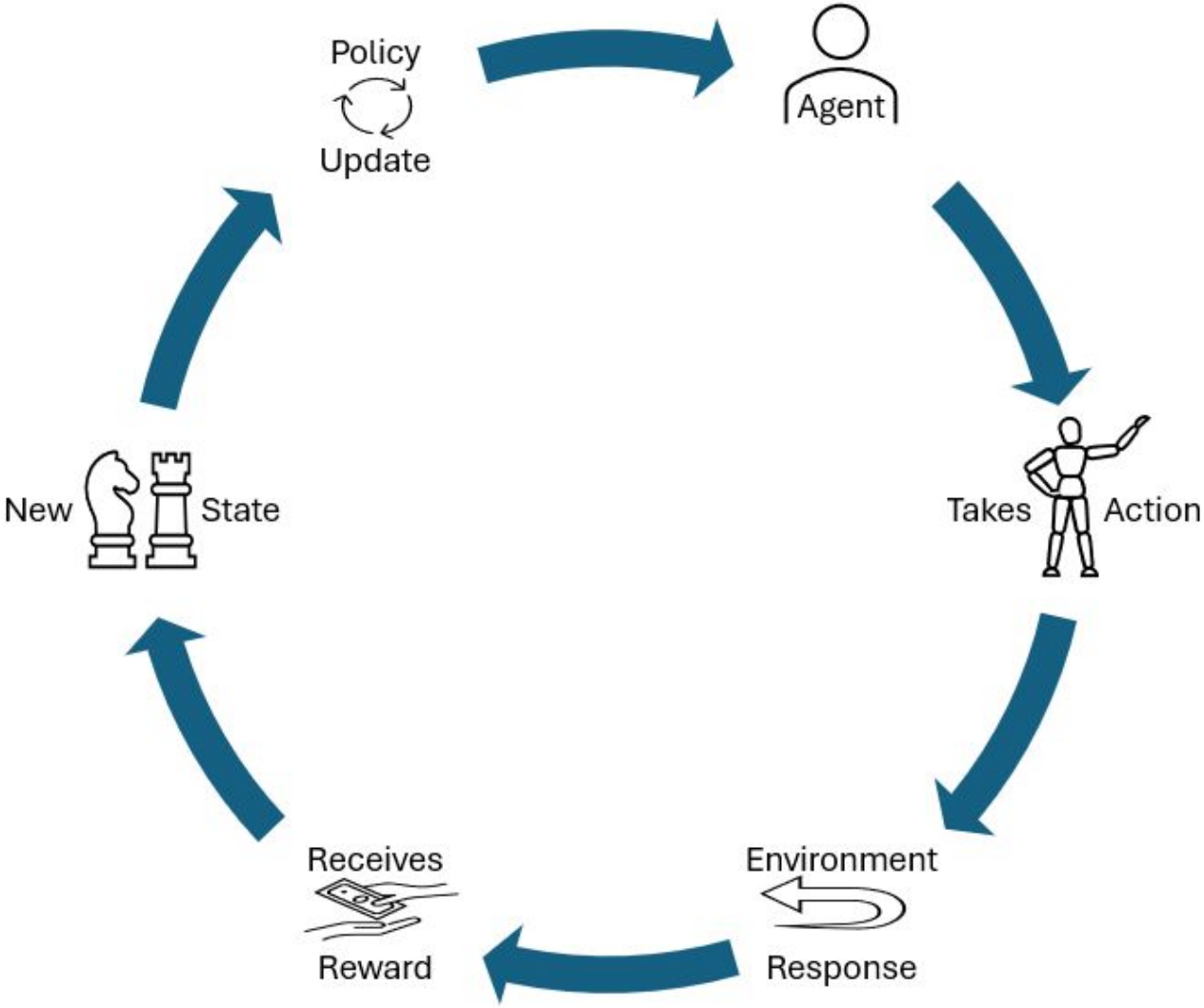
pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

print(pca.explained_variance_ratio_)
```

The Basics

- A different paradigm: an agent learns by interacting with an environment
- The agent takes actions, receives rewards (or penalties), and updates its strategy
- Goal: learn a policy that maximizes cumulative reward over time
- No labeled dataset — the agent learns through trial and error, not from examples




Policy
Update
Agent
Takes
Action
Environment
Response
Receives
Reward
New
State

Where It's Used

Domain	Example
Game playing	Agents that learn to play Chess, Go, and Atari games
Robotics	Learning to walk, grasp objects, and navigate
Recommendations	Learning which content keeps users engaged over time
Resource management	Data-center cooling, traffic-light control

SUMMARY

Key Takeaways

- Ensembles combine many models into one stronger model — bagging (Random Forest) reduces variance, boosting (XGBoost) reduces bias
- Feature importance explains a model's decisions; feature selection simplifies and speeds up the pipeline
- Hyperparameter tuning (grid or random search) squeezes out extra performance — always validate with cross-validation
- PCA compresses correlated features into fewer, information-dense components
- Reinforcement learning is a different paradigm entirely — agents learn from reward signals, not labeled data